EXPOSICIONES

ARQUITECTURA MONOLITICA

La arquitectura monolitica es un modelo de desarrollo el cual se efectua en capaz, esta arquitectura se basa en 4 capaz las cuales son: usuario, interfaz, modulos y resultado.
esta modelo se usa principalmente en proyectos pequeños ya que tiene una arquitectura muy simplificada.
su principal ventaja es tener un rendimiento muy efectivo, es decir muy rapido, y su principal desventaja es debido a asu gran despliegue que no soporta mucho codigo.

ARQUITECTURA EN CAPAZ

La arquitectura en capas es una forma de organizar un sistema de software dividiéndolo en diferentes niveles, donde cada capa tiene una responsabilidad específica.
ademas de esto, es usada comunmente para proyectos grandes.
Su principal ventaja es ser muy efectiva para trabajos en equipo.
Su desventajas son el tener mucha complejidad al iniciar el proyecto,otra de estas es que los cambios en una capa pueden afectar a otras si están demasiado acopladas

ARQUITECTUA CLIENTE - SERVIDOR

Esta arquitectura se basa en que el cliente solicite un servicio y el servidor de respuesta de ello. esta arquitectura tiene como base principal el usar 3 tipos diferentes de capaz las cuales son: 2 capaz: Estas capaz se basan en cliente servidor.
3 capaz: Estas capaz se basasn en cliente modelo servidor
N Capaz: Estas capaz se basan en presentación, lógica, cache, datos y almacenamiento.
Su principal ventaja es que muchos clientes pueden interactuar con este sistema y una de sus principales desventajas es que se puede bloquear el seridor y lo que provoca esto es perdida de datos

ARQUITECTURA MVC

Arquitectura MVC (Modelo - Vista - Controlador)
Esta arquitectura se basa en dividir una aplicación en 3 componentes principales, los cuales son:
Modelo: Se encarga de gestionar los datos y las reglas de negocio.
Vista: Se encarga de mostrar la información y la interfaz con la que interactúa el usuario.
Controlador: Se encarga de recibir las acciones del usuario y coordinar la comunicación entre el modelo y la vista.
Principal ventaja: Permite organizar el código, separar las responsabilidades y facilitar el mantenimiento y las modificaciones del sistema.
Principal desventaja: Puede aumentar la complejidad del desarrollo y generar más archivos y componentes, especialmente en proyectos pequeños.

ARQUITECTURA DE MICROSERVICIOS

Es un estilo de arquitectura que divide una aplicación en servicios pequeños e independientes, donde cada uno cumple una función del negocio y se comunica con los demás mediante APIs.

Principal ventaja: Cada servicio se puede escalar y desplegar por separado, y si uno falla el resto sigue funcionando.

Principal desventaja: Aumenta la complejidad de la comunicación, el monitoreo y el despliegue.

Patrones de diseño: Composite y Flyweight

ARQUITECTURA ORIENTADA A SERVICIOS (SOA)

Es un modelo que organiza el sistema como servicios reutilizables que se comunican por la red, normalmente mediante un bus de servicios (ESB) y protocolos como SOAP o REST.

Principal ventaja: Permite reutilizar servicios e integrar sistemas hechos con tecnologías diferentes.

Principal desventaja: El bus central puede volverse un cuello de botella y es más costoso de implementar.

Patrones de diseño: Observer y Mediator

ARQUITECTURA HEXAGONAL

Separa la lógica de negocio (el núcleo) del exterior mediante puertos (interfaces) y adaptadores (base de datos, interfaz web, APIs).

Principal ventaja: El núcleo no depende de las tecnologías, así que es fácil de probar y de cambiar.

Principal desventaja: Requiere más interfaces y código al inicio, lo que puede ser excesivo en proyectos pequeños.

Patrones de diseño: Strategy y State